Recurrent Small Language Models: Architecture, Training, and Evaluation

Jayant Dubey

October 2026

Abstract

This report consolidates a ten-part research programme on small, recurrent language models into a single volume. The empirical centerpiece is a loop-depth extrapolation study on a 44M-parameter recurrent mixture-of-experts transformer trained on TinyStories: test-time gains saturate at the trained loop depth and slightly degrade beyond it. Around this result, the report develops the architecture analysis (sparse routing, grouped-query attention, depth–width tradeoffs), training methodology (curriculum design, loop-aware distillation), and evaluation and deployment concerns (test-time compute taxonomy, quantization, mechanistic probes, and a benchmark proposal) for sub-billion-parameter recurrent models. Proposed experiments are labelled as such throughout; the only executed study is the loop sweep of Section 3.

1 Introduction

Small language models are having a moment. As the field's attention has concentrated on ever-larger systems, a parallel line of work asks what can be achieved under tight parameter budgets through *architectural* rather than *scale* innovation. Recurrent transformers—which iterate a shared block instead of stacking distinct layers—are among the most promising directions: they convert parameters into effective depth, offer a natural test-time compute knob in the loop count, and pair well with sparse mixture-of-experts layers.

This report grew from a single training run and the questions it raised. In October 2026 the author trained a 44M-parameter recurrent transformer (OpenMythos architecture) on TinyStories and, after training, swept the loop count at evaluation time. The result—gains saturate at the trained depth and slightly degrade beyond it—is a clean negative finding with a sharp implication: test-time loop extrapolation is not free. The ten studies collected here work outward from that finding, covering the architecture that produced it, how such models should be trained, and how they should be evaluated and deployed.

The report is organized as follows. Section 2 describes the reference model and training setup once, so that later chapters need not repeat it. Section 3 presents the empirical centerpiece, the loop-depth sweep. Section 4 analyses the architecture (routing, attention, and depth–width tradeoffs). Section 5 addresses training methodology (curriculum and distillation). Section 6 covers evaluation and deployment (test-time compute, quantization, mechanistic analysis, and a benchmark proposal). Section 7 concludes.

A note on honesty: with the exception of the loop sweep in Section 3, the studies in this report are analytical, methodological, or position pieces. Proposed experiments are labelled as proposed; no results are claimed that were not measured. The status of each chapter:

Section	Status
3. Loop-depth sweep	Executed (single run; 95% CIs in Section 3)
4. MoE routing	Analytical
5. Curriculum protocol	Proposed
6. Test-time taxonomy	Position
7. Quantization	Analytical
8. Distillation protocol	Proposed
9. Mechanistic probes	Proposed
10. TinyLoop-Bench	Design proposal

2 Reference Model and Training Setup

All studies in this report refer to a single reference configuration. It is described here once; later chapters cite this chapter rather than restating it.

Architecture. The model follows the OpenMythos design (Gomez, 2024): 43,974,722 parameters, model dimension 512, 6 unshared prelude layers and 2 unshared coda layers framing a shared recurrent block executed for 4 loops, 8 experts per mixture-of-experts feedforward layer, grouped-query attention, and a GPT-2 byte-pair vocabulary of 50,257 tokens. The prelude/coda structure is a descendant of the Universal Transformer (Dehghani et al., 2019) family, with parameter sharing in the spirit of ALBERT (Lan et al., 2020). Adaptive-depth variants allocate iteration per token rather than fixing it globally (Elbayad et al., 2020; Raposo et al., 2024); the reference model here fixes the loop count at training time, which is what makes the test-time extrapolation question sharp.

Training. The reference run trains on TinyStories (Eldan and Li, 2023) for 4,850 steps (120,004,608 tokens, approximately 152 minutes on a single Kaggle T4 GPU in fp16). Training loss falls from 9.5242 to 1.5263. Qualitative inspection shows a clear progression from incoherent output early in training to fluent multi-sentence stories by the end.

Attribution. The OpenMythos architecture is due to Gomez (Gomez, 2024) (GitHub repository, 2024). The 44M training run, the loop-depth sweep, and all analyses in this report are the author's own work; no released checkpoints are used.

3 Test-Time Loop Extrapolation: An Empirical Study

3.1 The Loop-Depth Sweep

3.1.1 Method

Evaluation protocol. After training, we freeze the weights and evaluate with loop counts $n \in \{1,2,4,8,12,16\}$ on 256 fresh TinyStories sequences of length 256. For each n we record mean cross-entropy loss and perplexity, and generate continuations from a fixed prompt for qualitative comparison. Sampling uncertainty is quantified with bootstrap 95% confidence intervals (10,000 percentile resamples over the 256 sequences); all paired comparisons below use per-sequence paired resampling. The $n = 4$ setting matches training; all others are pure test-time extrapolation or reduction.

3.1.2 Results

Table 1 reports the sweep. Three regimes are visible.

Recurrence helps up to the trained depth. Cutting loops below training is catastrophic for quality: a single loop yields perplexity 29.38, and doubling to 2 loops more than halves it to 11.18. The trained depth of 4 reaches 4.41. The recurrent iterations are doing substantial, non-redundant work.

Extrapolation yields no gain. Running 8 loops—twice the training count—*hurts* slightly (4.41 $\rightarrow$ 4.90), and further loops to 12 and 16 change nothing (4.91, 4.91). The dynamics do not continue to refine; they appear to converge to a fixed point near, but slightly worse than, the trained-depth output. The degradation is not sampling noise: the 95% bootstrap intervals on the two means do not overlap ([4.28, 4.55] at 4 loops vs. [4.75, 5.06] at 8 loops), and the paired per-sequence difference is $+0.106$ loss with interval $[+0.101, +0.110]$. The 8-to-12 difference, by contrast, is $+0.001$ loss—negligible in magnitude—confirming a genuine plateau rather than continued drift.

Loops n	1	2	4 (trained)	8	12	16
Loss	3.380	2.414	**1.484**	1.590	1.591	1.591
95% CI	[3.343, 3.419]	[2.384, 2.446]	**[1.454, 1.516]**	[1.559, 1.622]	[1.560, 1.624]	[1.560, 1.623]
Perplexity	29.38	11.18	**4.41**	4.90	4.91	4.91
95% CI	[28.30, 30.52]	[10.84, 11.54]	**[4.28, 4.55]**	[4.75, 5.06]	[4.76, 5.07]	[4.76, 5.07]

Table 1: Test-time loop sweep: mean loss and perplexity on 256 held-out TinyStories sequences (length 256). 95% confidence intervals are bootstrap percentile intervals (10,000 resamples over the 256 sequences). Bold marks the training depth.

Generations stay coherent. Qualitatively, samples at all loop counts are grammatical story continuations; over-looping does not cause collapse or repetition. The degradation is in token-level likelihood, not in catastrophic failure modes.

3.1.3 Discussion

Our central finding is that the recurrent dynamics are *calibrated* to the training iteration count. This is consistent with the model learning an approximate fixed-point iteration whose contraction properties hold for the trained horizon: within it, each loop refines; beyond it, the iteration has converged and additional steps add only noise.

This cautions against treating loop count as a free test-time scaling knob. If extrapolation is desired, it likely requires training-time support—e.g., randomized loop counts per batch, or an explicit halting objective—rather than emerging by default. Notably, our results do not contradict the broader test-time scaling literature: sampling-based methods (Snell et al., 2024) scale a different axis (number of attempts) than recurrent refinement (depth of a single attempt).

A limitation is scale: our study uses one 44M model on one dataset. Within that scope, sampling uncertainty is now quantified—the key 4-to-8-loop degradation survives a paired bootstrap test—but across-run uncertainty (different seeds, different data orderings) remains unmeasured and is the natural next experiment. Whether larger recurrent models, or training with loop-count randomization, recover extrapolation is left to future work. A second limitation is metric scope: perplexity may miss qualitative gains from extra loops on harder reasoning tasks, which TinyStories does not exercise.

3.1.4 Conclusion

On a 44M recurrent transformer trained at 4 loops, test-time loop extrapolation does not deliver free gains: quality saturates at the training depth and slightly degrades beyond it (4.41 $\rightarrow$ 4.90 perplexity at 8 loops) before plateauing. Recurrence is powerful within its trained horizon and yields nothing further outside it. We release our evaluation protocol to enable comparable sweeps at larger scales.

4 Architecture Analysis

4.1 Mixture-of-Experts Routing at Sub-100M Scale

Background

Top-k routing. For token representation x, a router computes logits $g = W_r x \in \mathbb{R}^E$ and probabilities $p = \mathrm{softmax}(g)$. With top-k routing, the token is dispatched to the k experts with largest p_i, weighted by their (renormalised) probabilities. The Switch Transformer (Fedus et al., 2022) popularised $k = 1$ with a load-balancing auxiliary loss; we keep k general and specialise where needed.

The auxiliary loss. Let f_i be the fraction of tokens in a batch dispatched to expert i and P_i the mean routing probability assigned to expert i. The Switch load-balancing loss is

$$\mathcal{L}_{\text{aux}} = \alpha E \sum_{i=1}^{E} f_i P_i, \tag{1}$$

with coefficient $\alpha > 0$. We derive its behaviour in Section 4.1.

Routing as estimation: the small-batch problem The router is a classifier whose "labels" are implicit: a token should go to the expert that will process it best. Its gradient at each step is estimated from the tokens routed to each expert. With N tokens per optimisation step and E experts under top-1 routing, token counts follow a multinomial distribution, so for expert i with routing share q_i,

$$\mathbb{E}[n_i] = Nq_i, \qquad \mathrm{Var}(n_i) = Nq_i(1-q_i). \tag{2}$$

Under uniform routing ($q_i = 1/E$), the relative standard deviation of expert i's token count is

$$\frac{\sqrt{\mathrm{Var}(n_i)}}{\mathbb{E}[n_i]} = \sqrt{\frac{E-1}{N}} \approx \frac{1}{\sqrt{N/E}}. \tag{3}$$

The router's gradient for expert i's parameters is an average over n_i tokens; its noise therefore scales roughly as $1/\sqrt{N/E}$ (the approximation is coarse at $E = 8$, off by about 6%). The quantity N/E—tokens per expert per step—is the router's effective sample size.

For our 44M configuration, $N \approx 120{,}004{,}608/4{,}850 \approx 2.47 \times 10^4$ tokens per step and $E = 8$, giving $N/E \approx 3{,}090$ tokens per expert per step under uniform routing, with relative fluctuation around 1.8%. That is comfortable *at the step level*. But two caveats bite at small scale. First, if gradient accumulation is used, the per-microbatch N can be an order of magnitude smaller, pushing relative fluctuations past 10%. Second, and more fundamentally, small models train on far fewer tokens in total—120M here against the trillions of large runs—so the router gets far fewer updates over training to average out the noise or to recover from an early bad lock-in. The router is thus solving a bandit problem with few pulls per arm per round and few rounds overall: the classic conditions for premature exploitation.

The rich-get-richer map Beyond noise, there is a deterministic feedback. Let $f^{(t)} \in \Delta^{E-1}$ be the routed fractions at step t and $\ell^{(t)}$ the corresponding mean router logits, with $f^{(t)} = \mathrm{softmax}(\ell^{(t)})$. Expert i's parameters move in proportion to the tokens it sees, so to first order its logit grows in proportion to its routed fraction:

$$\ell^{(t+1)} = \ell^{(t)} + \alpha f^{(t)}, \qquad \alpha > 0, \tag{4}$$

where α absorbs the learning rate, the tokens per step, and the mean per-token gradient alignment. The uniform point $\ell = c\mathbf{1}$ ($f_i = 1/E$) is a fixed point. Linearising around it in the zero-sum gauge ($\sum_i \varepsilon_i = 0$),

$$f_i \approx \frac{1}{E} + \frac{\varepsilon_i}{E}, \qquad \varepsilon' = \frac{\alpha}{E}\varepsilon. \tag{5}$$

Perturbations therefore grow geometrically whenever

$$\alpha > E. \tag{6}$$

Since $\alpha = \eta N \bar{g}$ (learning rate $\times$ tokens per step $\times$ mean per-token logit gradient), condition (6) is easier to satisfy when per-token gradients are large and coherent—precisely the early-training regime where some experts randomly fit the data better than others. The map is deliberately simple, but it isolates the mechanism: routing concentration is *self-amplifying*, and the uniform distribution is an unstable equilibrium once feedback exceeds the threshold. Noise (Section 4.1) decides *which* experts win; the feedback decides that *someone* wins everything.

The load-balancing loss, derived The auxiliary loss (1) penalises the correlation between dispatched fractions f_i and mean probabilities P_i. In the idealised case $f_i = P_i$,

$$\sum_{i=1}^{E} f_i P_i = \sum_{i=1}^{E} f_i^2 \geq \frac{1}{E}, \tag{7}$$

by convexity, with equality iff routing is uniform. The factor E normalises the minimum to 1, so $\mathscr{L}_{\text{aux}} \geq \alpha$ with equality at perfect balance. The loss is minimised by spreading tokens evenly—but note it does not distinguish *which* tokens go where, only the marginal distribution. It is a regulariser on the router's marginals, not a teaching signal for routing quality.

Choosing α. The auxiliary term contributes $O(\alpha)$ per token to the total loss. If α is too large, the router optimises balance at the expense of the language-modelling objective (uniform routing with random experts); if too small, the feedback of Section 4.1 dominates. A practical, scale-free heuristic: choose α so that $\mathscr{L}_{\text{aux}}$ is roughly 1–10% of the LM loss at initialisation, then verify the diagnostics of Section 4.1 rather than tuning blindly. The Switch Transformer's

$\alpha = 0.01$ (Fedus et al., 2022) sits in this range for typical setups, but the right value moves with E and batch size, so it should be re-derived per configuration, not copied.

Capacity, dropping, and utilisation In practice each expert accepts at most $C = \gamma \cdot kN/E$ tokens per batch ($\gamma \geq 1$ the capacity factor); overflow tokens are dropped (passed through unprocessed). Define utilisation $U_i = \min(1, n_i/C)$. Dropping interacts badly with collapse: once an expert is under-utilised, its effective batch shrinks further, its gradients get noisier, and recovery becomes less likely. A capacity factor γ moderately above 1 absorbs the multinomial fluctuation derived in Section 4.1 without masking systematic imbalance—dropping should be rare under healthy routing and common only under collapse, which makes the drop rate itself a diagnostic.

Proposed diagnostics We propose logging, per MoE layer and per evaluation window:

1. **Routing entropy.** $H = -\sum_i \bar{q}_i \log \bar{q}_i$ over corpus-level routing shares $\bar{q}_i$, normalised by $\log E$. Healthy: near 1. Collapsing: trending toward 0.
2. **Gini coefficient** of per-expert token counts. A single number in $[0, 1]$; 0 is perfect balance.
3. **Dead-expert fraction:** share of experts with $\bar{q}_i$ below $1/(10E)$ over the window. Nonzero is an alarm.
4. **Drop rate:** fraction of tokens dropped by capacity limits. Spikes indicate the imbalance has become structural.

The point of these metrics is early warning: by the time validation loss reflects collapse, the router has usually locked in and the starved experts cannot recover within a small model's token budget.

Discussion Applied to our setting—$E = 8$ experts, $\approx 2.5 \times 10^4$ tokens per step, 120M training tokens on TinyStories—the analysis suggests the risk is real but manageable: step-level token counts per expert are adequate, while microbatch-level noise and the short total training horizon are the threats. The prescriptions are unglamorous: keep α honest via the initialisation heuristic, log the four diagnostics from the first step, and treat any nonzero dead-expert fraction as a stop-and-fix event rather than a curiosity. Jitter on router logits (as in Switch) and a router-specific lower learning rate are compatible additions, but they treat symptoms; the diagnostics treat the disease by making it visible.

Limitations This paper is entirely analytical: we report no new experiments, and the dynamical map of Section 4.1 is illustrative, not a fitted model of any real router. We assume top-k routing with k small; expert-choice routing and learned temperature schedules are out of scope. TinyStories is a simple distribution; routing behaviour on more diverse corpora may differ quantitatively.

Conclusion Small MoE models are not small versions of large MoE models where routing is concerned. The router learns from N/E tokens per step, and at sub-100M scale that number, together with short training horizons, makes expert collapse a first-order risk rather than a curiosity. The instability condition $\alpha > E$, the auxiliary loss, and the four diagnostics give practitioners a complete, cheap toolkit: understand the feedback, regularise it, and watch it.

4.2 Grouped-Query Attention at Small Scale

Background

Attention variants. With model width d, h_q query heads and head dimension $d_h = d/h_q$: multi-head attention (MHA) uses $h_{kv} = h_q$ key/value heads; multi-query attention uses $h_{kv} = 1$; grouped-query attention (Ainslie et al., 2023) uses $1 < h_{kv} < h_q$, with each KV head shared by h_q/h_{kv} query heads. GQA was shown to match MHA quality closely while substantially beating single-head sharing on quality.

Parameter arithmetic Per attention layer, with biases ignored:

$$P_{\text{MHA}} = \underbrace{d^2}_{Q} + \underbrace{d^2}_{K} + \underbrace{d^2}_{V} + \underbrace{d^2}_{O} = 4d^2. \quad (8)$$

For GQA with h_{kv} KV heads, the K and V projections map $d \to h_{kv} d_h$:

$$P_{\text{GQA}}(h_{kv}) = 2d^2 + 2\,d\,h_{kv} d_h = 2d^2 \left(1 + \frac{h_{kv}}{h_q}\right). \quad (9)$$

With $d = 512$, $h_q = 8$, $d_h = 64$: MHA costs $4 \times 262{,}144 = 1{,}048{,}576$ parameters per layer. GQA with $h_{kv} = 2$ costs $524{,}288 + 131{,}072 = 655{,}360$ per layer, a saving of 393,216 parameters per layer (37.5% of the attention sub-layer). Across the 8 unique attention layers of our reference model (6 prelude + 2 coda), that is ≈ 3.15M parameters, or about 7.2% of the 43,974,722 total. A welcome discount, but not the reason to adopt GQA.

KV-cache arithmetic The KV cache stores, per token per layer, one key and one value vector per KV head. In fp16:

$$C_{\text{tok,layer}}(h_{kv}) = 2 \cdot h_{kv} \cdot d_h \cdot 2 \text{ bytes}. \quad (10)$$

For our dimensions: MHA ($h_{kv} = 8$) needs 2,048 bytes per token per layer; GQA with $h_{kv} = 2$ needs 512 bytes—a factor $h_q/h_{kv} = 4$ reduction, independent of model size. Total cache:

$$C_{\text{total}} = U \cdot C_{\text{tok,layer}} \cdot T \cdot B, \quad (11)$$

where U is the number of unrolled layers holding cache, T the context length and B the batch size. Note that in a looped architecture, each loop iteration produces its own KV entries, so U counts unrolled depth (prelude + loops × loop block + coda), not unique layers.

Worked serving scenarios Take $U = 12$ unrolled layers as an illustrative depth and fp16 throughout. Model weights occupy $\approx$ 88MB. All sizes below are decimal (1 MB $= 10^6$ bytes).

Scenario	B	T	MHA cache	GQA-2 cache
Single chat	1	512	12.6 MB	3.1 MB
Batched serving	8	2,048	403 MB	101 MB
Heavy batch	32	4,096	3.22 GB	0.81 GB

The pattern is stark. For a single short conversation, the MHA cache (12.6 MB) is a footnote next to 88 MB of weights—GQA buys almost nothing. At batch 8 and 2k context, the MHA cache (403 MB) is over 4× the weights, and GQA reclaims some 300 MB. At heavy batching it is the difference between fitting on one modest GPU and not fitting at all.

There is a second, throughput-shaped win: autoregressive decoding is memory-bandwidth bound, and each step streams the whole cache. GQA cuts bytes-per-step by h_q/h_{kv}, which translates directly into higher token throughput in cache-bound regimes—again, only where the cache is large enough to be the bottleneck.

Why training barely notices During training there is no KV cache; the K and V *activations* are recomputed per forward pass. GQA shrinks them by h_{kv}/h_q, but K/V activations are a small slice of total activation memory, which is dominated by $O(BTd)$ terms (Q, O, FFN intermediates, residuals) that GQA does not touch. Expect low-single-digit percent training memory savings and no meaningful throughput change: GQA is a serving optimisation, full stop.

Discussion: when GQA matters at small scale The decision rule falls out of Section 4.2: compute C_{total} for MHA in your serving regime; if it is comparable to or larger than the weight footprint, GQA (or even fewer KV heads) is worth it; if it is an order of magnitude smaller, it is cosmetic. Concretely for 44M–500M models:

- **On-device / single-user chat:** negligible. Skip it unless the parameter saving is wanted for its own sake.
- **Batched inference servers, long context:** decisive. The cache dominates and GQA's 4× cut (at $h_{kv} = 2$) directly buys batch size and context length.
- **Speculative or parallel decoding:** any technique that multiplies effective batch size pushes you into the cache-dominated regime sooner.

The quality cost, per Ainslie et al. (2023), is small at

these grouping ratios, so the trade is nearly free where it matters and unnecessary where it does not.

Limitations All figures assume fp16 and the stated head geometry; int8/int4 quantised caches scale all numbers down uniformly without changing the ratios. We analyse memory, not the engineering cost of implementing grouped attention kernels, which is real but one-off. Quality parity is taken from Ainslie et al. (2023) rather than re-measured here.

Conclusion GQA's value proposition does not fade at small scale—it *relocates*. The parameter saving ($\sim$7% at 44M) is a bonus; the KV-cache factor of h_q/h_{kv} is the main event, and it matters exactly when serving memory is cache-bound: large batches, long contexts. For short-context single-user inference it is negligible, and for training it is nearly invisible. Size the cache for your regime, then decide.

4.3 Depth–Width Tradeoffs in Looped Architectures

Background The Universal Transformer (Dehghani et al., 2019) replaced fixed depth with recurrent iteration of a shared block. ALBERT (Lan et al., 2020) showed cross-layer parameter sharing preserves quality while shrinking models. The OpenMythos family (Gomez, 2024) combines recurrence with sparse experts in a prelude/loop/coda structure. Test-time loop extrapolation—running extra loops at inference—is the natural test-time scaling axis for such architectures, though recent small-scale evidence finds gains saturating at the training loop count (Section 3.1).

The budget equations Let L_u be the number of unique non-looped layers (prelude + coda), L_b the unique layers in the looped block, R the loop count, d the width, and $p \approx 12d^2$ the parameters per dense layer (attention $4d^2$ plus feed-forward $8d^2$; MoE multiplies the feed-forward term by the expert count). Splitting off fixed costs (embeddings, output head, norms) as P_0,

$$P = P_0 + (L_u + L_b)\,p, \qquad D_{\text{eff}} = L_u + R L_b, \quad (12)$$

where D_{eff} is the *effective* (unrolled) depth: the number of layer evaluations per forward pass. Eliminating L_b,

$$D_{\text{eff}} = R\,\frac{P - P_0}{p} - (R-1)\,L_u. \quad (13)$$

Two readings of (13): at fixed R, effective depth grows with the parameter budget as usual; at fixed P, effective depth grows *linearly in* R. Loops manufacture depth out of thin parameter air.

The catch is compute. Forward FLOPs per token scale as $2\,p\,D_{\text{eff}}$ (plus small terms), and training as roughly $6\,p\,D_{\text{eff}}$. A dense model with P parameters spends $\approx 6P$ FLOPs per token; a looped model with the same P spends $6\,p\,D_{\text{eff}} > 6P$, because shared parameters are reused. *Loops do not save compute; they convert compute into depth at fixed parameter count.* Any Chinchilla-style compute-optimality analysis must be re-derived for looped models, since parameters and FLOPs no longer move together.

The conditioning constraint Depth bought with loops has a price beyond FLOPs: gradient flow. Backpropagation through R repetitions of the loop block multiplies by its Jacobian J at each pass, so gradient norms scale roughly as ρ^R, where ρ is the spectral radius of J. If $\rho > 1$, gradients explode with R; if $\rho < 1$ they vanish and the later loops learn nothing. Stable, useful looping requires $\rho \approx 1$—the block must act as a near-isometry that still does computational work each iteration.

This yields a structural argument for *small* loop blocks. A one- or two-layer operator is a tighter dynamical system: fewer interacting nonlinearities, easier to keep near-isometric, and a cleaner fixed-point-iteration semantics (refine the representation, don't rebuild it). A deep looped block must simultaneously learn a complex transformation *and* keep its R-fold composition well-conditioned—a harder joint problem. Hence our first hypothesis:

Hypothesis H1. At fixed P and fixed training compute, configurations with small L_b and moderate R outperform large L_b with $R = 1$, because the conditioning constraint binds before the expressivity constraint.

The prelude and coda play different roles from the loop block: the prelude maps tokens into the latent

space the loop refines, and the coda maps back to logits. Neither is iterated, so neither faces the ρ^R constraint, but starving them bottlenecks everything the loops can do. Hence:

Hypothesis H2. There is a minimal viable L_u (we conjecture 4–8 layers at $d = 512$ scale); below it, loop-count increases cannot compensate, because the prelude/coda mappings themselves lack capacity.

Width, loops, and the compute knob With L_u floored by H2 and L_b kept small by H1, the remaining budget goes to width d and loop count R. Width buys representational capacity per step; loops buy sequential refinement steps. They are not interchangeable: width helps single-pass pattern matching, loops help iterative computation. This suggests treating R not as an architectural constant but as a *compute knob* akin to test-time compute (Snell et al., 2024):

Hypothesis H3. The optimal R scales with task difficulty (reasoning depth required), not with model size. A model trained with randomised loop counts should therefore extrapolate to R beyond its mean training value, whereas fixed-R training calibrates the dynamics to exactly R—which is consistent with loop-sweep measurements showing perplexity saturating precisely at the training loop count (29.38 at 1 loop $\rightarrow$ 4.41 at 4 loops, then flat) on our 44M model.

H3 reframes the negative extrapolation result as expected behaviour rather than a failure: fixed-R training teaches the block a 4-step operator, and there is no reason a 4-step operator iterated 8 times should improve.

Proposed experiments We propose, but do not run: (E1) a fixed-P, fixed-compute sweep over (L_u, L_b, R) on TinyStories (Eldan and Li, 2023), measuring validation loss, to test H1/H2 directly; (E2) tracking the spectral radius ρ of the loop-block Jacobian through training, testing whether stable runs indeed hold $\rho \approx 1$ and whether instability predicts divergence; (E3) training with R sampled per batch and sweeping R at evaluation, testing H3's extrapolation claim.

Limitations Entirely theoretical: no training runs are reported and the hypotheses are untested. The ρ^R argument treats the loop block Jacobian as fixed, ignoring its evolution during training and input dependence. TinyStories exercises limited reasoning depth, so H3's difficulty-scaling claim needs harder benchmarks to bite.

Conclusion Parameter sharing splits the depth question in two: unique layers, bounded by the budget equation (12), and effective depth, purchasable with loops at the price of compute and conditioning. The analysis points to small loop blocks, a minimal but non-trivial prelude/coda, width for capacity, and loop count as a difficulty-matched compute knob. Whether the hypotheses survive contact with experiment is now an empirical question—the equations say exactly which experiments to run.

5 Training Methodology

5.1 Curriculum Strategies for TinyStories

Background

Curriculum learning Bengio et al. (2009) introduced curriculum learning by analogy with human teaching: present easy examples first, then gradually harder ones. Follow-up work formalised this as a pacing function $c(t)$, the fraction of the (difficulty-sorted) data available at training step t, growing from a small $c(0)$ to $c(T) = 1$. Reported benefits cluster in two regimes: faster early convergence, and better final performance on tasks with a clear difficulty structure. Reported failures cluster where difficulty is ill-defined or where the pacing interacts badly with the learning-rate schedule. Both the promise and the failure modes motivate preregistration: the difficulty metric and pacing must be fixed before training, or the experimenter can always find *some* ordering that "worked" after the fact.

TinyStories as a testbed TinyStories contains short children's stories generated by larger models under tight vocabulary and narrative constraints (Eldan and Li, 2023). Three properties matter for our purposes.

First, stories vary genuinely in difficulty: some use only the simplest words and one character, others use rarer vocabulary and multi-step plots. Second, the corpus is fully available offline, so corpus-wide statistics (unigram frequencies, length distributions) are computable exactly. Third, the reference 44M run gives us a calibrated baseline: we know what shuffled training achieves at this budget, so curriculum conditions have a fixed target to beat.

Story-complexity metrics We define five metrics, each computable from the story text alone plus corpus-wide unigram counts. All are defined before any training.

Vocabulary rarity. For a story S with tokens $w_1, \ldots, w_n$ and corpus unigram probabilities $p(\cdot)$,

$$R(S) = -\frac{1}{n} \sum_{i=1}^{n} \log p(w_i). \tag{14}$$

Stories built from rare words score high. This is the mean unigram surprisal, cheap and stable.

Mean sentence length. Longer sentences in children's stories correlate with subordinate clauses and heavier syntax. Computed in tokens after the same BPE tokenisation used in training.

Narrative depth. A proxy for plot complexity:

$$\begin{aligned} N(S) = {} & \#\text{distinct named characters} \\ & + \#\text{causal/temporal connectives}, \end{aligned} \tag{15}$$

where connectives are drawn from a fixed list (*because, so, then, after, before, when, but*). Crude, but transparent and reproducible.

Lexical diversity. Type–token ratio over the story. High diversity at fixed length indicates richer word choice.

Story length. Total tokens. Longer stories demand longer-range coherence.

Difficulty score. Each metric is z-scored across the corpus, and the difficulty score is the unweighted mean of the five z-scores:

$$D(S) = \frac{1}{5} \sum_{m=1}^{5} z_m(S). \tag{16}$$

Equal weights are a deliberate, preregistered choice. Tuning weights after seeing results would convert the study into a fishing expedition; the weights are therefore fixed before any training run begins.

Proposed experimental protocol

Fixed factors Model, tokeniser, optimiser, learning-rate schedule, batch size, and total token budget (120M) are identical to the reference run in every condition. Each condition is trained from three random seeds. Only the order in which stories are presented differs.

Conditions

(a) **Shuffled baseline.** Uniform random order, as in the reference run.

(b) **Easy-to-hard, linear pacing.** Stories sorted by $D(S)$; pacing $c(t) = c_0 + (1 - c_0)\,t/T$ with $c_0 = 0.1$.

(c) **Easy-to-hard, root pacing.** Same ordering, but $c(t) = \sqrt{(1 - c_0^2)\,t/T + c_0^2}$. This schedule is concave: it rises steeply at the start, so harder material is introduced *earlier* than under linear pacing—testing whether early exposure to difficulty accelerates learning rather than destabilising it.

(d) **Anti-curriculum.** Hard-to-easy with linear pacing. A control: if *any* non-random ordering helped, this condition would catch it.

(e) **Two-stage.** First half of training on the easiest 50% of stories (shuffled within), second half on the full corpus shuffled. Tests whether a brief "foundation" phase suffices.

Evaluation We track training loss continuously and evaluate held-out perplexity at eight evenly spaced checkpoints plus the final model, on a fixed validation split never used for pacing decisions. Beyond perplexity, we use two targeted probes. A *grammaticality probe*: synthetic minimal pairs (correct vs. agreement-violated sentences in the TinyStories vocabulary) scored by relative likelihood. A *narrative-coherence probe*: fixed story beginnings completed by each model and rated for coherence, either by human raters or by a fixed LLM judge with a published rubric. Perplexity measures fit; the probes measure what the fit bought.

Hypotheses

H1. Easy-to-hard conditions (b, c) reach lower final held-out perplexity than the shuffled baseline (a) at the fixed token budget.

H2. Root pacing (c) outperforms linear pacing (b): earlier exposure to harder material accelerates representation learning rather than destabilising early training.

H3. The anti-curriculum (d) underperforms the baseline (a), confirming that ordering effects are directional rather than mere artefacts of non-random presentation.

H4. Curriculum gains concentrate in the narrative-coherence probe rather than raw perplexity: ordering teaches structure more than it teaches token statistics.

Each hypothesis is falsifiable as stated. In particular, if (b) and (c) fail to beat (a), the honest conclusion is that curricula are not worth the engineering at this scale—a useful negative result.

Discussion

Why this testbed can settle the question. Most curriculum studies inherit an ambiguity: is the difficulty metric real, or did the experimenter choose one that flatters the method? Here the metrics are mechanical, the weights preregistered, and the corpus synthetic enough that the metrics track something genuine. A null result here would be evidence, not noise.

Confounds. Two deserve attention. First, pacing interacts with the learning-rate schedule: easy data early plus a warmup schedule may double-count the same effect. We keep the schedule fixed across conditions precisely so this interaction, if present, is held constant rather than tuned away. Second, difficulty sorting changes the token distribution seen early in training, which changes early gradient statistics; three seeds per condition give a rough sense of the variance this introduces.

What success would mean. A clean H1–H2 win would argue for curriculum as a free improvement to small-model pretraining: no extra tokens, no extra parameters, only ordering. It would also motivate testing whether the effect persists at larger budgets, where the relative cost of sorting is negligible.

Limitations This paper proposes; it does not execute. Until the runs exist, the hypotheses are untested. TinyStories is synthetic, so any finding may not transfer to natural corpora where difficulty is harder to define. The difficulty score's equal weights are arbitrary by construction—a principled weighting would itself require a study. Finally, all conditions share one architecture; curriculum effects may interact with model structure (notably the recurrent loops of our reference model) in ways a single-architecture study cannot see. The three-seed design proposed here has limited power to separate nearby curricula: with only three runs per condition, small but real ordering effects may not reach significance, so a null result should be read as inconclusive rather than as evidence of no effect.

Conclusion We have specified a complete, preregistered experimental programme for testing curriculum learning in TinyStories pretraining at the 44M scale: five computable difficulty metrics, one fixed difficulty score, five training conditions, and four falsifiable hypotheses. The protocol is designed so that both positive and null results are informative. The next step is execution.

5.2 Distilling into Recurrent Students

Background

Knowledge distillation. Hinton et al. (2015) introduced training a student on the teacher's softened output distribution, transferring "dark knowledge" about class similarities. Sanh et al. (2019) extended this with hidden-state matching between aligned layers, producing a widely used compact BERT.

Recurrent transformers. The Universal Transformer (Dehghani et al., 2019) replaces fixed depth with a shared block applied recurrently. The OpenMythos variant frames the recurrent core with unshared prelude layers (input processing) and coda layers (output processing). Our reference student configuration, matching the 44M model studied in a companion paper of this series, uses dimension 512, 6 prelude plus 2 coda layers, 4 loops of a shared block, 8 experts per MoE layer, grouped-query attention, and a 50,257-token BPE vocabulary (43,974,722 parameters total), trained on TinyStories (Eldan and Li, 2023).

The calibration problem. In the companion study, the 44M model's test-time loop sweep showed perplexity 29.38 at 1 loop falling to 4.41 at the trained depth of 4 loops, then degrading slightly to 4.90 at 8 loops and plateauing through 16 loops. The recurrent dynamics are calibrated to the training iteration count; extra test-time loops do not refine further. Any distillation protocol for recurrent students must reckon with this: supervision that fixes a single loop count risks baking in the same calibration.

Why Naive Distillation Under-Constrains Recurrence Consider a teacher T with L distinct layers and a student S whose shared block f_θ is iterated K times between prelude P and coda C:

$$h_0 = P(x), \quad h_k = f_\theta(h_{k-1}) \ (k = 1 \ldots K), \quad y = C(h_K). \tag{17}$$

Naive distillation minimises a divergence $D(p_T(y \mid x) \,\|\, p_S(y \mid x))$ over the final outputs only. The difficulty is degrees of freedom: for fixed P, C, and output y, the intermediate sequence $(h_1, \ldots, h_{K-1})$ is unconstrained. In particular, the degenerate solution $f_\theta \approx$ identity satisfies any output-matching objective while performing zero iterative refinement; all computation migrates to P and C, which the optimiser prefers because they receive undiluted gradients. The student then matches the teacher's logits with a recurrent block that does nothing—and, per the calibration finding above, its loop count becomes a frozen hyperparameter rather than a meaningful compute knob.

A second, subtler failure: even when f_θ is nontrivial, nothing in the objective requires successive iterations to implement *progressive* refinement. The block may learn oscillatory or redundant dynamics that happen to land near the teacher's output at exactly step K. Attention-map inspection in such students (proposed in §5.2) would be expected to show near-identical patterns across loops, in contrast to a teacher whose layer-wise attention evolves from local to global structure.

Loop-Aware Distillation Protocol We propose supervising the *trajectory*, not just the endpoint.

Layer-to-loop alignment. Partition the teacher's L layers into K contiguous groups $G_1, \ldots, G_K$. For loop k, define the target representation as the teacher's output after group G_k:

$$\mathscr{L}_{\text{hid}} = \sum_{k=1}^{K} \lambda_k \left\| h_k - \phi_k\left(T_{G_k}(x)\right) \right\|^2, \tag{18}$$

where $T_{G_k}(x)$ is the teacher hidden state after group k and ϕ_k is a learned linear projection bridging dimensionality gaps. The weights λ_k follow a ramp $\lambda_k \propto k/K$, encoding the prior that later iterations should carry more of the teacher's high-level computation.

Cross-iteration attention transfer. For each loop k and each attention head, we match the student's attention distribution to the mean attention map of teacher group G_k via KL divergence:

$$\mathscr{L}_{\text{attn}} = \sum_{k=1}^{K} \mu_k \, \mathrm{KL}\left(\bar{A}^T_{G_k} \,\|\, A^S_k\right). \tag{19}$$

This directly pressures the recurrent block to reproduce the teacher's layer-wise attention evolution—from local patterns early to long-range structure late—rather than collapsing to static maps.

Loop-count curriculum. To avoid calibrating the dynamics to a single K, training randomises the executed loop count per batch, $K \sim \mathscr{U}\{1, K_{\max}\}$, with the

alignment targets recomputed for the sampled depth (teacher groups re-partitioned accordingly). Early training uses small $K_{\max}$, growing to the full depth—a curriculum over iteration count. Final-logit matching against the teacher is retained throughout:

$$\mathscr{L} = \alpha\mathscr{L}_{\text{task}} + \beta\mathscr{L}_{\text{logit}} + \gamma\mathscr{L}_{\text{hid}} + \delta\mathscr{L}_{\text{attn}}, \qquad (20)$$

with γ, δ warmed up from zero once the student matches the teacher's outputs coarsely (we propose α=1, β=1, γ, δ ramping to 0.5).

Proposed Experiments

Setup. Teacher: a standard 12-layer transformer (≈120M parameters, proposed configuration) trained on TinyStories (Eldan and Li, 2023). Student: the 44M recurrent configuration above ($K_{\max}$=4). All runs proposed; no results are claimed.

Experiment 1: ablation of supervision terms. Train four students: (a) naive (β only); (b) +$\mathscr{L}_{\text{hid}}$; (c) +$\mathscr{L}_{\text{attn}}$; (d) full protocol. Report perplexity at each loop count 1…8 and the loop-sweep curve shape. *Predicted* outcome: (a) matches teacher perplexity at exactly 4 loops but shows flat or degraded behaviour off the trained depth with static cross-loop attention; (d) shows monotonic improvement with loop count and evolving attention maps. These are hypotheses to be tested, not findings.

Experiment 2: iteration-content probes. For each student, train linear probes on h_k to predict the next token, and measure per-loop probe accuracy. A student doing genuine iterative refinement should show rising probe accuracy across k; a degenerate student will show flat probe curves with all gains in the coda.

Experiment 3: loop-count generalisation. Evaluate each student at $K \in \{1, 2, 4, 8, 12, 16\}$. The curriculum-trained student is hypothesised to degrade gracefully rather than exhibiting the sharp calibration seen in fixed-K training.

Discussion If the protocol works, it yields a genuinely new capability: small recurrent students that inherit a large teacher's knowledge *and* implement it as iterative refinement, with loop count as a working test-time knob. If per-loop supervision proves unnecessary—if naive distillation already produces refining dynamics—that is itself an informative negative result about how much structure output matching imposes.

A limitation is teacher choice: distilling from a fixed-depth teacher may transfer depth-shaped computation that recurrence can only approximate. An alternative worth testing is a recurrent teacher, which would make the alignment exact but sacrifices the abundance of strong pretrained fixed-depth teachers.

Limitations This is a method proposal without empirical validation. The predicted outcomes in §5.2 are hypotheses. Scale is limited to the sub-100M regime and TinyStories; whether loop-aware distillation helps at larger scales or on reasoning-heavy data is open. The alignment between teacher layer groups and student loops is a heuristic—there may be no faithful mapping, in which case the per-loop targets could harm more than help.

Conclusion Distilling into recurrent students requires supervising the trajectory, not just the endpoint. We have specified a loop-aware protocol—per-loop hidden targets, cross-iteration attention transfer, and a loop-count curriculum—together with the experiments that would test it. The question it answers is precise: can distillation produce recurrent students whose loops do real work?

6 Evaluation and Deployment

6.1 A Taxonomy of Test-Time Compute

Background

Test-time scaling at large scale Snell et al. (2024) study how to spend a fixed inference FLOPs budget across strategies: repeated sampling with majority vote or best-of-N selection, and sequential revision guided by verifiers. Two findings matter here. First, for easier questions, parallel strategies (best-of-N, self-consistency) are compute-optimal: indepen-

dent samples explore the answer space cheaply. Second, for harder questions, sequential strategies win: iterative revision steered by a verifier outperforms parallel sampling per FLOP. The crossover depends on task difficulty and verifier quality. Both findings assume verifiers worth having—a safe assumption at 8B, a loaded one at 44M.

Sampling-based methods Best-of-N draws N independent completions and keeps the best according to a scorer. Self-consistency (Wang et al., 2023) draws N reasoning paths and takes the majority answer, needing no learned scorer at all. Both multiply inference cost by N (modulo batching) and both live or die by selection: best-of-N needs a verifier better than chance; self-consistency needs the model's modal answer to be the right one, which holds for reasoning tasks and fails for open-ended generation.

Verifier-based methods Outcome verifiers score final answers; process verifiers score intermediate steps (Cobbe et al., 2021). They enable search (beam search, MCTS, revision loops) and reranking. The large-scale literature treats the verifier as a sunk cost—train once, reuse forever. At small scale the verifier is a second model to train, store, and serve, and its quality is bounded by the same data and capacity constraints as the policy it judges.

Recurrent depth Recurrent transformers iterate a shared block for a fixed number of loops during training (Dehghani et al., 2019). Architectures in this family, including OpenMythos (Gomez, 2024), invite an obvious test-time trick: run *more* loops at inference than were used in training. Unlike sampling, this costs no extra parameters and no verifier. Whether the recurrent dynamics actually improve with extra iterations is an empirical question, which Section 3.1 answers for one concrete model.

A three-axis taxonomy We classify test-time methods along three axes, each with a cost model in units of base-model forward passes per token, F.

Axis I: sampling. Best-of-N, self-consistency, majority vote. Cost $\approx N \cdot F$, reducible by batching but not asymptotically. Requires a selection mechanism: a verifier (Axis III) or a majority signal. Failure mode: when the verifier is weak, best-of-N selects confidently wrong answers—it amplifies verifier error, not just policy error.

Axis II: depth. Extra recurrent loops, adaptive computation time, early-exit inversion (running *past* the trained depth). Cost $\approx L \cdot F$ for L loops. Requires nothing but the model itself. Failure mode: the dynamics may be calibrated to the training iteration count, in which case extra loops add compute without adding quality—or subtract it.

Axis III: verification. Learned outcome or process reward models, rerankers, revision critics. Cost $= V + N \cdot F$ where V is the verifier's forward cost per candidate. This axis rarely stands alone; it is the steering wheel for Axes I and II. Failure mode: at small scale, V is not negligible, and a weak verifier misteers.

The axes compose: Snell's sequential revision is Axis II$\times$III; best-of-N with a reward model is Axis I$\times$III. The taxonomy's point is to make the cost of each composition explicit before asking which compositions a 44M budget can afford.

What changes below a billion parameters

The verifier overhead problem Consider a learned verifier at half the policy's size—generous by large-scale standards, where verifiers are often far smaller relative to the policy. For an 8B policy, a 4B verifier is a 50% serving overhead. For a 44M policy, a 22M verifier is also a 50% overhead, but the absolute quality of a 22M verifier trained on the same limited data is far lower: verifier quality scales with capacity too, and there is no large-scale teacher to distil from cheaply. The relative cost is the same; the absolute quality is not. This breaks the large-scale assumption that verification is cheap and good.

Sampling arithmetic Best-of-8 on a 44M model spends $8 \times 44\text{M} = 352\text{M}$ parameters' worth of forward compute per token—roughly the cost of a single

greedy pass from a 350M model. The honest comparison is therefore not "44M vs. 44M with best-of-8" but "best-of-8 from 44M vs. greedy from 350M." If the 350M model wins that comparison, sampling is a worse use of the FLOPs than training a bigger model. Whether it does is task-dependent and, at small scale, largely unmeasured—which is itself the point: the large-scale literature rarely runs this comparison because at 70B the alternative (a 560B model) does not exist.

Self-consistency escapes the verifier problem but keeps the sampling cost, and its majority-vote mechanism assumes the model's modal answer is correct. On TinyStories-style generation, where there is no single right answer, the mechanism has nothing to grip.

Depth: measured, not assumed Here we have direct evidence. Our 44M-parameter recurrent transformer (43,974,722 parameters; dim 512; 6 prelude + 2 coda layers; 4 training loops; 8 experts; GQA; trained on TinyStories for 120M tokens, loss 9.5242 → 1.5263) was evaluated with loop counts swept from 1 to 16 over 256 held-out sequences (full results in Section 3.1, Table 1).

Recurrence helps enormously up to the training depth—perplexity falls by a factor of 6.7 from 1 to 4 loops—and then the gains stop dead. Eight loops are slightly *worse* than four; sixteen add nothing. Generations remain coherent at all depths, so this is not collapse; it is saturation. The recurrent dynamics are calibrated to the iteration count they were trained with. Test-time depth extrapolation, the cheapest axis in the taxonomy, is not free after all—at least not without training-time support such as randomised loop counts.

The memory wall A final small-scale constraint: test-time methods that hold N candidates or deep activation caches multiply memory, not just FLOPs. At 44M the weights fit comfortably anywhere, which tempts wasteful inference schemes; but the deployment targets for small models are phones and browsers, where memory bandwidth—not capacity—is the bottleneck, and N-way sampling spends it N times over.

Position: three bets for small-scale test-time compute

1. **Bet on trained-depth recurrence, not extrapolated depth.** Running the loops you trained is pure win (our 1→4 loop result); running more is wishful thinking without training-time support. Research should target *adaptive* depth within the trained range—early exit for easy tokens—rather than extrapolation beyond it.

2. **Bet on cheap, honest verifiers.** Learned reward models are too expensive and too weak at this scale. Heuristic verifiers—length penalties, repetition detectors, grammar checkers, task-specific validators—cost near zero and fail transparently. A verifier you can read is better than a verifier you can only hope is calibrated.

3. **Sample only with a trustworthy selector.** Best-of-N is justified at small scale only when the selection signal is strong: exact-match verifiers for constrained tasks, or self-consistency on tasks with verifiable answers. Unverifiable open-ended generation should not be sampled—the selector cannot tell good from lucky.

Open questions: can randomised training loop counts unlock genuine extrapolation? At what parameter count does a learned verifier become worth its serving cost? And does the Snell easy/hard crossover replicate when both policy and verifier are small?

Limitations This is a position paper; the taxonomy is argued, not experimentally validated as a whole. The depth-axis evidence is a single model on a single corpus—one careful data point, not a law. Snell's findings may transfer better than we argue, particularly the easy-question sampling results. We also ignore fine-tuning-time analogues (e.g., distilling test-time compute back into weights), which may be the most practical small-scale strategy of all.

Conclusion Test-time compute is real, but its economics are scale-dependent. Sampling multiplies cost against a fixed budget, learned verifiers cost a second model the budget cannot spare, and recurrent depth—the one free axis—saturates at the training depth. The small-scale research agenda should stop

importing large-scale recipes and start designing test-time methods for the regime where the verifier must cost less than the model it serves.

6.2 Quantization Robustness of Small MoE Models

Background

Uniform affine quantization Post-training quantization typically maps each weight x to a b-bit integer via an affine transform with scale s and zero-point z:

$$x_q = \mathrm{clip}\left(\mathrm{round}\left(\frac{x}{s}\right) + z,\ 0,\ 2^b - 1\right), \qquad \hat{x} = s\,(x_q - z), \tag{21}$$

with $s = (\max x - \min x)/(2^b - 1)$ over a calibration range. For weights roughly uniform within the range, the mean squared reconstruction error is approximately $s^2/12$—quadratic in the range. Two consequences follow that matter below: error grows with the weight range, and per-group scales (small s per group) beat per-tensor scales whenever ranges vary across groups. GPTQ (Frantar et al., 2023) improves on round-to-nearest by correcting each quantized weight against the inverse Hessian of the layer's inputs; LLM.int8() (Dettmers et al., 2022) isolates outlier feature dimensions in higher precision; SmoothQuant (Xiao et al., 2023) migrates quantization difficulty from activations to weights. All three were developed on dense models at billion-plus scale.

Sparse mixture-of-experts In a sparse MoE layer, each token's hidden state h is routed by a small linear map to logits $r = W_r h$ over E experts; a top-k operation selects the experts that process the token, and their outputs are combined by softmax weights (Shazeer et al., 2017). Only $k \ll E$ experts activate per token, so capacity scales with E while compute scales with k. The router W_r is tiny—in our reference model, $d_{\text{model}} = 512$ and $E = 8$ give $512 \times 8 = 4{,}096$ parameters—but it controls the fate of millions of expert parameters.

Reference model Our running example is a 44M-parameter recurrent MoE transformer: 43,974,722 parameters total, hidden dimension 512, 6 prelude and 2 coda layers, 4 recurrent loops, 8 experts per MoE layer, grouped-query attention (Ainslie et al., 2023), GPT-2 BPE vocabulary of 50,257, trained on TinyStories (Eldan and Li, 2023) for 120M tokens (loss 9.5242 → 1.5263). We also compute footprints for the 133M (133,334,082 parameters) and 500M (507,581,506 parameters) configurations in the same family.

Why MoE quantizes differently

Per-expert calibration skew In a dense layer, every weight serves every token, so calibration data is the full distribution. In MoE, expert e is calibrated only on the tokens routed to it. Routing is deliberately skewed—load-balancing is approximate, and some experts inevitably specialise narrowly—so experts see different token counts, different activation statistics, and different effective weight ranges. Since quantization error scales with s^2 and s with the range, rarely-fired experts calibrated on thin data carry noisier scales and larger reconstruction error:

$$\mathscr{E}_e = \frac{\|W_e - \hat{W}_e\|_F^2}{\|W_e\|_F^2} \tag{22}$$

should vary systematically with expert utilisation, not randomly. The prediction: per-expert relative error anti-correlates with expert firing frequency. A dense model has no such structure; its error is approximately uniform across the layer.

Router precision and the top-k discontinuity The router is 4,096 parameters out of 44 million—under 0.01%—and it is the worst place in the model to quantize aggressively. The reason is the top-k operator: it is discontinuous. A perturbation δ in router logits that merely rescales softmax probabilities in a dense model can *flip* expert selection here, rerouting the token to an entirely different expert and changing the output by $O(1)$, not $O(\delta)$. Formally, where the dense sensitivity is Lipschitz in the weights, the MoE sensitivity has jump discontinuities at every logit tie. The practical prescription is mixed precision by construction: keep the router (and its input normalisation) at 16 bits regardless of what happens elsewhere. The cost is negligible—8 KB—and the protection is structural.

The embedding table dominates This is the single most important quantitative fact about quantizing small language models, MoE or otherwise. The embedding table has $50{,}257 \times 512 = 25{,}731{,}584$ parameters: **58.5% of the entire 44M model**. More than half the weights are a lookup table. Embedding quantization behaves differently from matmul quantization: there is no accumulation of products, only selection, so the error model is per-row rather than per-multiply—but the rows for rare tokens are updated rarely during training and may carry noisier values, mirroring the expert-skew argument. The standard practice of keeping embeddings at higher precision is well motivated here twice over: they are most of the model, and their rows are unevenly trained.

Shared dense versus expert parameters The prelude and coda layers are dense and shared across all tokens; their quantization error is the familiar uniform kind. The MoE layers hold most non-embedding parameters and carry the structured sensitivities above. A sound quantization scheme therefore treats three parameter classes differently—dense trunk, experts, router—not one.

Memory arithmetic at three scales Weight memory in megabytes (decimal) for the three configurations:

Model	FP32	FP16	INT8	INT4
44M	175.9	88.0	44.0	22.0
133M	533.3	266.7	133.3	66.7
500M	2030.3	1015.2	507.6	253.8

Sizes in MB. The 44M reference model has 43,974,722 parameters.

These are exact multiplications of the parameter counts; they exclude scale-factor overhead, which for per-group INT4 (group size 128, FP16 scales) adds roughly $P/128 \times 2$ bytes—about 0.3 MB at 44M, negligible.

A concrete mixed-precision proposal. Keep the embedding table (25.7M params) and router at FP16; quantize everything else to INT4:

$$\underbrace{25.7\text{M} \times 2\,\text{B}}_{51.5\,\text{MB}} + \underbrace{18.2\text{M} \times 0.5\,\text{B}}_{9.1\,\text{MB}} \approx 60.6\,\text{MB}, \tag{23}$$

versus 88.0 MB for uniform FP16—a 31% saving that spends precision exactly where the router-discontinuity argument argued above says it matters. Whether the INT4 experts hold up is precisely what the protocol below is designed to test.

A note on the KV cache: grouped-query attention already compresses it, and cache quantization is orthogonal to weight quantization—we scope this paper to weights.

Proposed evaluation protocol We propose the following measurements, none yet performed. For each precision setting (FP16 baseline, INT8 uniform, INT4 uniform, and the mixed-precision scheme above):

1. **Held-out perplexity** on a fixed TinyStories validation split, reported per checkpoint-equivalent (post-training quantization needs no retraining, so this is one number per setting).

2. **Per-expert relative error** $\mathscr{E}_e$ as defined above, correlated against expert firing frequency measured on the validation set. Hypothesis: strong negative correlation.

3. **Router flip rate:** the fraction of validation tokens whose top-k expert set changes between FP16 and quantized router,

$$F = \frac{1}{T}\sum_{t=1}^{T} \mathbf{1}\{\text{topk}(\tilde{r}_t) \neq \text{topk}(r_t)\}. \tag{24}$$

   Hypothesis: F stays near zero when the router is held at FP16 and jumps discontinuously when it is quantized—the signature of the router-discontinuity argument developed in Section 4.

4. **Generation spot checks:** fixed prompts completed at each precision, judged for coherence degradation. Catches failures that perplexity averages hide.

The protocol is designed so that a null result is informative: if per-expert error shows no correlation with utilisation, the calibration-skew argument is wrong, and uniform quantization is safer for MoE than we claim.

Discussion The small-MoE case is the interesting one. In a trillion-parameter MoE, experts are redundant—losing one to quantization noise is absorbed by the others. At 44M with 8 experts, every expert is load-bearing; there is no redundancy to hide behind. This inverts the usual intuition that small models are easier to quantize: the *weights* are fewer, but the *structure* is more fragile. The embedding dominance compounds this: most of what you are quantizing is a lookup table, not compute.

The broader lesson is that quantization schemes should be architecture-aware. Per-tensor INT8 treats a router logit and an expert weight as the same kind of number; they are not.

Limitations This paper is analytical: no quantization experiment has been run, and every sensitivity claim is a prediction, not a measurement. The memory numbers are exact arithmetic, but real deployments add runtime, activation, and cache overheads we do not model. Our expert-skew argument assumes imperfect load balancing, which holds in practice but is a property of training, not of the architecture. Finally, we scope to weight quantization; activation and KV-cache quantization at small scale deserve their own analysis.

Conclusion Small sparse-MoE models have a quantization profile distinct from dense models: skewed per-expert calibration, a discontinuous router that demands high precision despite its size, and an embedding table that is most of the model. We derived these sensitivities, computed exact memory footprints from 22 MB to 2 GB across three scales, proposed a mixed-precision scheme at 60.6 MB for the 44M model, and specified a protocol—including a router flip-rate metric—that can confirm or refute the analysis. The measurements are the obvious next step.

6.3 What Do Extra Loops Compute?

Introduction The loop sweep is the most informative cheap experiment you can run on a recurrent language model. For our 44M-parameter OpenMythos model (43,974,722 parameters; dim 512; 6 prelude + 2 coda layers; 4 loops; 8 experts; GQA; trained on TinyStories for 120M tokens, loss 9.5242→1.5263), the sweep over 256 held-out sequences gives (full results in Section 3.1, Table 1):

Two facts stand out. First, the gains are enormous and concentrated: one extra loop (1→2) more than halves perplexity; the trained depth reaches 4.41. The iterations are doing substantial, non-redundant work. Second, the dynamics saturate hard: doubling past the trained depth slightly hurts, then nothing changes at all through 16 loops—yet generations stay grammatical and coherent throughout, so this is not collapse, it is convergence.

The natural next question—*what is each loop computing?*—has barely been asked for recurrent language models. Feedforward interpretability has tools for this: induction heads (Elhage et al., 2021) localise in-context learning to specific heads; the Universal Transformer literature (Dehghani et al., 2019) speculated about iterative refinement but rarely measured it per-iteration. We propose to turn the probe toolkit onto the loop dimension itself.

Hypotheses We state three testable hypotheses, each tied to the observed sweep.

H1: Coarse-to-fine refinement. Early loops resolve local choices (lexicon, agreement, short-range syntax); later loops handle long-range coherence (entity consistency, discourse structure). Under H1, the 29.38→11.18 drop (loop 1→2) should be dominated by local-likelihood improvements, while the 11.18→4.41 drop (loop 2→4) should show gains concentrated on tokens whose predictability depends on distant context. This is testable by stratifying per-token loss improvements by dependency length—a measurement, not a speculation.

H2: Fixed-point convergence. Beyond the trained depth, the recurrent block converges to an approximate fixed point: $h_k \approx h_{k-1}$ for $k > 4$, which explains both the plateau (4.90→4.91→4.91) and the preserved coherence (the fixed point is a sensible representation, just not a better one). Under H2, representation similarity between consecutive loops should approach 1.0 for $k \geq 8$ while remaining well below 1.0 for $k \leq 4$.

H3: Progressive circuit formation. Attention circuits that implement in-context behaviours—induction heads in the sense of Elhage et al. (2021)—form progressively across loops rather than appearing fully formed. Under H3, induction-head scores (prefix-matching behaviour in attention patterns) should rise monotonically over loops 1–4 and then stabilise, mirroring the perplexity curve.

Note the tension H1 and H2 jointly create, which is exactly what makes this interesting: the same block must implement *changing* computation early (refinement) and *stationary* computation late (convergence). Characterising the transition is the core empirical target.

Proposed Probe Protocol All experiments below are proposed. The model, data (256 held-out TinyStories sequences of length 256), and loop settings $\{1,2,4,8,12,16\}$ are fixed from the companion study; only the measurements are new.

P1: Per-loop representation similarity. For each sequence, record the recurrent block's output h_k at each loop k. Compute centred kernel alignment (CKA) and mean cosine similarity between (h_k, h_{k+1}) for $k = 1 \ldots 15$ at fixed total depth 16. *Predicted:* similarity well below 1.0 for $k < 4$, rising sharply, and $\approx$1.0 for $k \geq 8$ (H2). A control—similarity between h_k at matched loop index across different total depths (e.g. loop 2 of a 4-loop run vs loop 2 of a 16-loop run)—tests whether loop-indexed computation is depth-invariant.

P2: Attention-pattern evolution. For each loop, compute induction-head scores per head following Elhage et al. (2021) (attention paid to the token following a previous occurrence of the current token). Track the distribution of scores across loops. *Predicted:* median induction score rises over loops 1–4 and flattens thereafter (H3); heads that score highly should also show the largest cross-loop attention-map change early and the smallest late.

P3: Per-loop early-exit classifiers. Freeze the model; train a linear probe on each h_k to predict the next token, and record probe perplexity per loop. Additionally, train probes for auxiliary targets: part-of-speech of the next token (local syntax) and the identity of the story's protagonist (global coherence). *Predicted:* next-token probe perplexity tracks the observed 29.38→4.41 curve; the syntax probe saturates by loop 2 while the protagonist probe keeps improving to loop 4 (H1).

P4: Loop interventions. At fixed total depth 16: (a) skip loop k (replace h_k with h_{k-1}) for each k and measure the perplexity delta—this prices each iteration's marginal contribution; (b) freeze attention to its loop-4 pattern for all $k > 4$ and test whether the 4.90 plateau persists, distinguishing representational from attentional convergence.

What Would Change Our Minds The protocol is designed to be falsifiable. If P1 shows high similarity even between loops 1 and 2, H2's convergence story is wrong and something else—perhaps the coda, not the recurrent block—explains the gains. If P3's probes are flat across loops while output perplexity still drops 29.38→4.41, the refinement is happening outside the probed representations (prelude or coda), which would redirect the investigation. If induction scores (P2) are already maximal at loop 1, then in-context circuitry is established immediately and later loops do something orthogonal to the induction-head picture. Each outcome narrows the hypothesis space; that is the point.

Discussion Why does this matter beyond one 44M model? If recurrent refinement is real and characterisable—if we can say "loop 2 fixes syntax, loop 4 fixes discourse"—then loop count becomes an *interpretable* compute knob rather than a blind hyperparameter. It would also inform distillation into recurrent students (companion paper on loop-aware distillation, this series): per-loop supervision targets should match what each loop is actually for. Conversely, if the probes show the block converging to a fixed point with no structured per-loop division of labour, that reframes recurrence as learned depth with shared parameters—valuable, but not "refinement" in any mechanistic sense, and the test-time extrapolation story needs rethinking.

Limitations This paper proposes; it does not report. The hypotheses are grounded in one model's behavioural signature on one dataset (TinyStories), and synthetic children's stories may not exercise the circuitry (e.g. induction heads) that matters at scale. Probes are correlational: rising probe accuracy does not prove the model *uses* the probed information. The companion behavioural numbers are real; everything about mechanism here is conjecture awaiting the protocol.

Conclusion Perplexity $29.38 \to 4.41 \to 4.91$ across loops is a behavioural fact in search of a mechanistic explanation. We have posed the explanation as three falsifiable hypotheses and specified the four-probe protocol that would test them. The loops are doing something; it is time to look at what.

6.4 TinyLoop-Bench: A Benchmark Proposal

Introduction Benchmarks shape fields. GLUE told us what "understanding" meant for an era; perplexity leaderboards told us what fluency meant. For recurrent language models there is no equivalent instrument. A researcher who trains a looped transformer today reports the numbers available—perplexity at the trained loop count—and the loop-specific questions go unanswered: does quality improve monotonically with loops? Where does it saturate? How does a 44M recurrent model compare, per parameter, against a 44M fixed-depth model?

A companion study in this series ran exactly one such evaluation: a test-time loop sweep on a 44M OpenMythos model (43,974,722 parameters; dim 512; 6 prelude + 2 coda layers; 4 loops; 8 experts; GQA; GPT-2 BPE 50,257; TinyStories, 120M tokens, loss 9.5242→1.5263), measuring perplexity at 1, 2, 4, 8, 12, and 16 loops. The result—a sharp drop to 4.41 at the trained depth, then a flat plateau—was obtained with ad-hoc scripts. TinyLoop-Bench exists to make that evaluation standard, repeatable, and comparable across models.

Design Principles

1. **Loops are first-class.** Every metric is reported as a function of loop count, not just at the trained depth. A model card without a loop-scaling curve is incomplete.

2. **Cheap to run.** Sub-1B models exist so that iteration is fast; the suite must run on a single consumer GPU in under an hour. No task may require more than 512 evaluation sequences.

3. **Proposed, versioned, frozen.** The suite ships with a version number and frozen evaluation scripts; results cite the version. This paper specifies v0.1.

4. **Honest about construct validity.** TinyStories-scale data (Eldan and Li, 2023) measures fluency and basic coherence, not reasoning. The suite claims nothing beyond its constructs.

Task Categories

C1: Fluency. Standard language modelling on held-out TinyStories sequences (256 sequences $\times$ 256 tokens, fixed split, seed-pinned). Metric: perplexity at each loop count $n \in \{1, 2, 4, 8, 12, 16\}$. This is the baseline everything else builds on.

C2: Coherence over long generations. The model generates continuations of 256 tokens from 64 fixed prompts. Two proposed metrics:

- *Entity consistency*: named entities introduced in the prompt are extracted (by a fixed, open NER pass—proposed implementation: spaCy `en_core_web_sm`, pinned version); the score is the fraction still referenced consistently (no contradictory attributes) in the continuation. Reported per loop count.

- *Repetition rate*: fraction of generated 8-grams that repeat a previous 8-gram within the continuation. Lower is better; reported per loop count.

Both are heuristic proxies, documented as such; the suite reports them alongside, never instead of, human-readable samples (5 fixed prompts, outputs published verbatim in the report).

C3: Loop-scaling curves. The signature category. For each model, the suite records the full perplexity-vs-loops curve and derives three summary statistics (definitions proposed here):

- *Refinement gain* $G = \mathrm{PPL}(1)/\mathrm{PPL}(K_{\text{train}})$: how much iteration helps at all.
- *Saturation point* n^*: the smallest n with $\mathrm{PPL}(n) \leq 1.02 \times \min_m \mathrm{PPL}(m)$—where gains effectively stop.
- *Extrapolation slope* $E = \mathrm{PPL}(2K_{\text{train}}) - \min_m \mathrm{PPL}(m)$: positive means over-looping hurts, near zero means it plateaus, negative means genuine extrapolation.

For the companion 44M model these would read (illustrative, from the companion study's real numbers): $G = 29.38/4.41 \approx 6.7$, $n^* = 4$, $E = 4.90 - 4.41 = +0.49$—refinement helps enormously, saturates exactly at the trained depth, and extrapolation slightly hurts. The suite's job is to make triples like $(6.7, 4, +0.49)$ the standard way recurrent models are compared.

C4: Parameter-efficiency frontiers. Each submitted model is plotted against fixed-depth baselines at matched parameter counts on the C1–C3 metrics: quality per parameter. The suite ships reference baselines (proposed: 44M and 130M fixed-depth transformers trained on the same TinyStories split with the same token budget—to be trained when the suite is built). A recurrent model "wins" its frontier if it dominates the baseline curve at equal or fewer parameters.

Reference Evaluation Protocol

Environment. Single GPU, fp16 or bf16, batch size chosen to fit 12 GB VRAM; the harness auto-selects. All randomness seeded; seeds published.

Procedure. (1) Load checkpoint and config; the harness reads the trained loop count K_{train} from config. (2) Run C1 at $n \in \{1, 2, 4, 8, 12, 16\}$ (capped at $4K_{\text{train}}$ if larger). (3) Run C2 generations at $n \in \{1, K_{\text{train}}, 2K_{\text{train}}\}$. (4) Compute C3 statistics from the C1 curve. (5) Emit a JSON report plus a human-readable model card following the fixed template (see below).

Reporting template. Every submission reports: model identifier, parameter count, architecture summary (dim, layers, loops, experts, attention type, vocabulary), training tokens and dataset, the full C1 table, the (G, n^*, E) triple, C2 scores, C4 frontier position, suite version, and the five verbatim generations. No cherry-picking: the harness generates the card, not the author.

Harness Design (Proposed) A single Python package, `tinyloop-bench`, with three parts: *tasks* (dataset loaders, generation loop, metric functions), *runners* (a model interface requiring only `generate(prompt, n_loops)` and `perplexity(text, n_loops)`), and *reports* (JSON + Markdown card). The model interface is deliberately minimal so that any recurrent implementation—OpenMythos-style (Gomez, 2024) or otherwise—can plug in. Reference datasets are content-addressed by SHA-256; the exact TinyStories split and prompts ship with the package.

Discussion The most likely objection is scope: TinyStories is toy data, and a benchmark built on it cannot speak to reasoning or knowledge. We accept this and bound the claims accordingly—but note that the loop-scaling question is *cleanest* at small scale, where full sweeps are cheap and confounders are few. A suite that standardises (G, n^*, E) at 44M creates the vocabulary in which larger-scale recurrent evaluation can later be discussed.

A second risk is Goodhart drift: once (G, n^*, E) becomes a target, architectures may optimise the triple rather than genuine refinement (e.g. by making loop 1 artificially bad to inflate G). The verbatim generations and the C4 frontier are partial mitigations; ultimately, versioned frozen tasks plus community scrutiny are the defence, as with every benchmark.

Limitations This paper specifies a benchmark; it does not build or run one. The metric definitions (entity consistency, G, n^*, E) are proposals awaiting validation—in particular, whether entity consistency

as defined correlates with human judgements of coherence is untested. Reference baselines do not yet exist. The suite covers only decoder-only recurrent transformers on English story data in v0.1.

Conclusion Recurrent models need recurrent benchmarks. TinyLoop-Bench v0.1 specifies four task categories, concrete metrics including the loop-scaling triple (G, n^*, E), a frozen evaluation protocol, and a harness design—everything needed to build it. The next step is building it.

7 Conclusion

The throughline of this report is a single, stubborn empirical fact: on the reference 44M model, recurrence helps enormously within its trained horizon and not at all outside it. Everything else here is an attempt to understand, exploit, or work around that fact.

The architecture chapters show *why* the model looks the way it does: sparse routing concentrates learning signal per expert and demands careful load balancing at small batch sizes; grouped-query attention is a serving-time optimization whose training-time footprint is negligible at this scale; and the looped structure converts parameters into effective depth at the cost of gradient-conditioning constraints that favour small, near-isometric recurrent blocks.

The training chapters propose how to do better next time: curricula that order TinyStories by measurable complexity, and distillation protocols that supervise the recurrent dynamics themselves rather than only their final outputs. The evaluation chapters map the territory around the model: which test-time scaling axes are viable below a billion parameters, how quantization interacts with sparsity, what the loops might mechanistically be computing, and how the field should benchmark recurrent small models at all.

The most important next step is the one the centerpiece demands: training with randomized loop counts to test whether extrapolation can be *taught* rather than hoped for—no experiment in this report tests that yet. Until then, the honest summary of this programme is that test-time loop scaling, on the evidence of one 44M run, offers no free gains beyond the trained depth—and establishing that is itself progress.

References

References

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebron, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv:2305.13245*, 2023.

Bengio, Y., Louradour, J., Collobert, R., and Weston, J. Curriculum learning. In *ICML*, 2009.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv:2110.14168*, 2021.

Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., and Kaiser, Ł. Universal transformers. In *ICLR*, 2019.

Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *NeurIPS*, 2022.

Elbayad, M., Gu, J., Grave, E., and Auli, M. Depth-adaptive transformer. In *ICLR*, 2020.

Eldan, R. and Li, Y. TinyStories: How small can language models be and still speak coherent English? *arXiv:2305.07759*, 2023.

Elhage, N., Nanda, N., Olsson, C., Henighan, T., Joseph, N., et al. A mathematical framework for transformer circuits. *Anthropic*, 2021.

Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23:1–39, 2022.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *ICLR*, 2023.

Gomez, K. OpenMythos: recurrent mixture-of-experts transformers. *GitHub repository*, 2024.

Hinton, G., Vinyals, O., and Dean, J. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.

Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., and Soricut, R. ALBERT: A lite BERT for self-supervised learning of language representations. In *ICLR*, 2020.

Raposo, D., Ritter, S., Richards, B., Lillicrap, T., Humphreys, P., and Santoro, A. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv:2404.02258*, 2024.

Sanh, V., Debut, L., Chaumond, J., and Wolf, T. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv:1910.01108*, 2019.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*, 2017.

Snell, C., Lee, J., Xu, K., and Kumar, A. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv:2408.03314*, 2024.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *ICLR*, 2023 (arXiv:2203.11171).

Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *ICML*, 2023.